

## PLAN DE PARCHES — EcoRoute

### Calendario y Proceso para Actualizaciones de Dependencias

## 1. INTRODUCCIÓN

### 1.1 Propósito

Este documento define el proceso formal, el calendario y los criterios de decisión para gestionar las actualizaciones de dependencias del sistema EcoRoute. Su objetivo es garantizar que el entorno de ejecución del sistema se mantenga seguro, reproducible y compatible a lo largo del ciclo de vida del proyecto, minimizando el riesgo de regresiones por actualizaciones no controladas.

### 1.2 Alcance

El plan cubre todas las dependencias del sistema EcoRoute declaradas en los archivos `environment.yml` y `requirements.txt`, incluyendo el simulador SUMO, el framework PyTorch, las bibliotecas de análisis de datos y las herramientas de prueba.

### 1.3 Principio Base

EcoRoute fija las versiones de todas sus dependencias críticas. Ninguna dependencia se actualiza de forma automática o sin pasar por el proceso definido en este documento. La reproducibilidad científica del proyecto depende de que el entorno permanezca estable entre ejecuciones.

## 2. INVENTARIO DE DEPENDENCIAS

### 2.1 Dependencias Críticas

Son aquellas cuya actualización puede afectar los resultados del entrenamiento, la compatibilidad con SUMO o la reproducibilidad de los experimentos.

| ID     | Dependencia | Versión actual    | Rol en el sistema                        | Nivel de criticidad |
|--------|-------------|-------------------|------------------------------------------|---------------------|
| DEP-01 | Python      | 3.10.x            | Lenguaje base del sistema                | Crítico             |
| DEP-02 | SUMO        | 1.18.x            | Simulador de tráfico urbano              | Crítico             |
| DEP-03 | TraCI       | Incluido con SUMO | Interfaz de control del simulador        | Crítico             |
| DEP-04 | PyTorch     | 2.0.x             | Framework de red neuronal del agente DQN | Crítico             |

|        |              |        |                                               |         |
|--------|--------------|--------|-----------------------------------------------|---------|
| DEP-05 | Gymnasium    | 0.29.x | Interfaz estándar del entorno de RL           | Crítico |
| DEP-06 | NumPy        | 1.24.x | Operaciones vectoriales sobre estados y lotes | Crítico |
| DEP-07 | CUDA Toolkit | 11.8.x | Aceleración GPU para entrenamiento            | Crítico |

## 2.2 Dependencias de Análisis y Visualización

| ID     | Dependencia | Versión actual | Rol en el sistema                             | Nivel de criticidad |
|--------|-------------|----------------|-----------------------------------------------|---------------------|
| DEP-08 | pandas      | 2.0.x          | Registro y exportación de métricas            | Alto                |
| DEP-09 | matplotlib  | 3.7.x          | Gráficas de convergencia y comparativas       | Alto                |
| DEP-10 | seaborn     | 0.12.x         | Mapas de calor de densidad vehicular          | Medio               |
| DEP-11 | PyYAML      | 6.0.x          | Carga del archivo de configuración            | Alto                |
| DEP-12 | SciPy       | 1.11.x         | Pruebas estadísticas (Mann-Whitney U, t-test) | Alto                |

## 2.3 Dependencias de Desarrollo y Pruebas

| ID     | Dependencia | Versión actual | Rol en el sistema                  | Nivel de criticidad |
|--------|-------------|----------------|------------------------------------|---------------------|
| DEP-13 | pytest      | 7.4.x          | Ejecución de pruebas automatizadas | Medio               |
| DEP-14 | pytest-cov  | 4.1.x          | Reporte de cobertura de código     | Medio               |
| DEP-15 | black       | 23.x           | Formateador de código Python       | Bajo                |
| DEP-16 | flake8      | 6.x            | Analizador estático de código      | Bajo                |

## 3. CLASIFICACIÓN DE ACTUALIZACIONES

Toda actualización de dependencia se clasifica en una de las siguientes categorías antes de procesarla:

| Tipo                         | Descripción                                                | Ejemplo               | Acción requerida                                 |
|------------------------------|------------------------------------------------------------|-----------------------|--------------------------------------------------|
| <b>Parche de seguridad</b>   | Corrección de vulnerabilidad crítica (CVE)                 | NumPy 1.24.1 → 1.24.2 | Aplicar en menos de 7 días                       |
| <b>Parche de corrección</b>  | Corrección de bug sin cambio de API                        | PyTorch 2.0.0 → 2.0.1 | Aplicar en el siguiente ciclo mensual            |
| <b>Actualización menor</b>   | Nueva funcionalidad compatible con versión actual          | pandas 2.0 → 2.1      | Evaluar en ciclo trimestral                      |
| <b>Actualización mayor</b>   | Cambio de API o comportamiento potencialmente incompatible | PyTorch 2.0 → 3.0     | Evaluar en ciclo semestral con pruebas completas |
| <b>Actualización de SUMO</b> | Cualquier cambio en la versión de SUMO                     | SUMO 1.18 → 1.19      | Proceso especial (ver sección 5)                 |

## 4. PROCESO DE ACTUALIZACIÓN

### 4.1 Flujo General

Todo parche sigue este flujo sin excepción, independientemente de su tipo:

#### 1. DETECCIÓN

Identificar la nueva versión disponible (manual o via pip-outdated)

#### 2. CLASIFICACIÓN

Determinar el tipo de actualización (seguridad / corrección / menor / mayor)

#### 3. EVALUACIÓN DE IMPACTO

Revisar changelog oficial de la dependencia

Identificar cambios de API que afecten a EcoRoute

#### 4. RAMA DE PRUEBA

Crear rama Git: patch/dep-[nombre]-[version]

Actualizar versión en environment.yml y requirements.txt

#### 5. EJECUCIÓN DE PRUEBAS

Ejecutar suite completa de pytest

Verificar reproducibilidad con seed=42

Comparar métricas de entrenamiento con versión anterior

## 6. VALIDACIÓN

¿Todas las pruebas pasan? ¿Métricas estables?

SÍ → Continuar NO → Revertir y documentar el bloqueo

## 7. INTEGRACIÓN

Merge a rama principal

Actualizar environment.yml, requirements.txt y este documento

## 8. REGISTRO

Documentar en el Registro de Parches Aplicados (sección 7)

### 4.2 Criterios de Aprobación de un Parche

Para que un parche sea aprobado e integrado, debe cumplir todos los siguientes criterios:

| Criterio                | Condición de aprobación                                                                  |
|-------------------------|------------------------------------------------------------------------------------------|
| Suite de pruebas        | 100% de pruebas unitarias y de integración pasan                                         |
| Reproducibilidad        | Dos corridas con seed=42 producen métricas idénticas tras el parche                      |
| Convergencia del agente | La curva de pérdida TD no muestra regresión significativa respecto a la versión anterior |
| Cobertura de código     | La cobertura global no disminuye respecto al valor previo al parche                      |
| Compatibilidad SUMO     | TraCI conecta y opera correctamente con la versión de SUMO activa                        |

### 4.3 Criterios de Rechazo y Bloqueo

Un parche se rechaza y la dependencia se mantiene en su versión actual si:

| Causa de rechazo                                                          | Acción                                                             |
|---------------------------------------------------------------------------|--------------------------------------------------------------------|
| Una o más pruebas fallan tras la actualización                            | Revertir versión; documentar el fallo en el registro de bloqueos   |
| La reproducibilidad se ve afectada (seed=42 produce resultados distintos) | Revertir versión; investigar la fuente de aleatoriedad introducida |
| La API de TraCI cambia y rompe la integración con SUMO                    | Revertir versión; abrir issue para adaptar el wrapper TraCI        |

|                                                     |                                                         |
|-----------------------------------------------------|---------------------------------------------------------|
| La convergencia del agente se degrada más de un 10% | Revertir versión; investigar causa antes de re-intentar |
|-----------------------------------------------------|---------------------------------------------------------|

## 5. PROCESO ESPECIAL PARA ACTUALIZACIONES DE SUMO

Las actualizaciones de SUMO requieren un proceso más riguroso debido a que afectan simultáneamente al simulador, a la interfaz TraCI y a los modelos de emisiones:

| Paso | Actividad                                                                                             |
|------|-------------------------------------------------------------------------------------------------------|
| 1    | Revisar el changelog oficial de SUMO en <a href="https://sumo.dlr.de/changes">sumo.dlr.de/changes</a> |
| 2    | Identificar cambios en la API de TraCI que afecten a <code>sumo_env.py</code>                         |
| 3    | Identificar cambios en los modelos de emisiones (HBEFA, PHEMlight)                                    |
| 4    | Crear entorno conda paralelo con la nueva versión de SUMO                                             |
| 5    | Ejecutar suite completa de pruebas en el entorno paralelo                                             |
| 6    | Ejecutar entrenamiento completo de 100 episodios y comparar métricas con versión anterior             |
| 7    | Si todo es correcto: actualizar <code>environment.yml</code> y documentar                             |
| 8    | Mantener el entorno con la versión anterior disponible durante 30 días como respaldo                  |

## 6. CALENDARIO DE REVISIÓN

### 6.1 Revisiones Periódicas Programadas

| Frecuencia             | Actividad                                                                     | Dependencias revisadas |
|------------------------|-------------------------------------------------------------------------------|------------------------|
| Semanal                | Verificar CVEs críticos publicados para dependencias del proyecto             | Todas                  |
| Mensual                | Revisar parches de corrección disponibles y aplicar los aprobados             | DEP-04 a DEP-12        |
| Trimestral             | Evaluar actualizaciones menores de todas las dependencias                     | Todas                  |
| Semestral              | Evaluar actualizaciones mayores y nueva versión de SUMO                       | DEP-01 a DEP-07        |
| Al cierre del proyecto | Congelar versiones definitivas en <code>environment.yml</code> para archivado | Todas                  |

## 6.2 Calendario de Revisiones del Proyecto EcoRoute

| Revisión | Período | Tipo                            | Dependencias prioritarias             |
|----------|---------|---------------------------------|---------------------------------------|
| REV-01   | Mes 1   | Inicial — establecer línea base | Todas las DEP                         |
| REV-02   | Mes 2   | Mensual                         | DEP-04 PyTorch, DEP-06 NumPy          |
| REV-03   | Mes 3   | Mensual + Trimestral            | Todas las DEP de análisis             |
| REV-04   | Mes 4   | Mensual                         | DEP-08 pandas, DEP-12 SciPy           |
| REV-05   | Mes 5   | Mensual                         | DEP-09 matplotlib, DEP-13 pytest      |
| REV-06   | Mes 6   | Mensual + Semestral             | Evaluación de SUMO 1.19 y PyTorch 2.1 |

## 7. REGISTRO DE PARCHES APLICADOS

*Esta tabla se completa a medida que se aplican parches durante el desarrollo.*

| ID Parche | Fecha | Dependencia | Versión anterior | Versión nueva | Tipo | Pruebas ejecutadas | Resultado | Aplicado por |
|-----------|-------|-------------|------------------|---------------|------|--------------------|-----------|--------------|
| PAR-001   | —     | —           | —                | —             | —    | —                  | Pendiente | —            |

## 8. REGISTRO DE BLOQUEOS

*Esta tabla registra las actualizaciones evaluadas y rechazadas.*

| ID Bloqueo | Fecha | Dependencia | Versión evaluada | Causa del rechazo | Acción tomada | Revisión pendiente |
|------------|-------|-------------|------------------|-------------------|---------------|--------------------|
| BLQ-001    | —     | —           | —                | —                 | —             | —                  |

## 9. HERRAMIENTAS DE APOYO

| Herramienta                   | Uso                                       | Comando                      |
|-------------------------------|-------------------------------------------|------------------------------|
| <b>pip list --outdated</b>    | Detectar dependencias pip desactualizadas | pip list --outdated          |
| <b>conda update --dry-run</b> | Simular actualización conda sin aplicarla | conda update --all --dry-run |

|                  |                                                                 |                  |
|------------------|-----------------------------------------------------------------|------------------|
| <b>pip-audit</b> | Detectar vulnerabilidades CVE en dependencias pip               | pip-audit        |
| <b>safety</b>    | Verificar dependencias contra base de datos de vulnerabilidades | safety check     |
| <b>pytest</b>    | Validar que el parche no rompe el sistema                       | pytest tests/ -v |